NULL

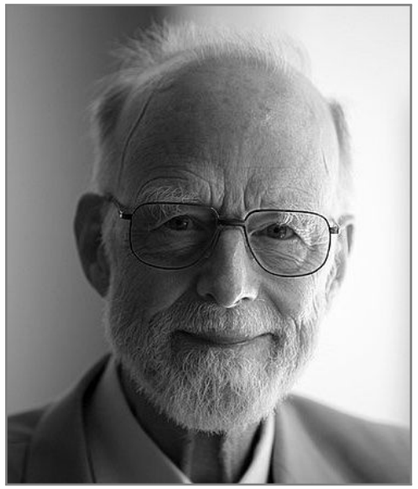
Fail Fast, Returning, Checking, Objects

Yegor Bugayenko

Lecture #6 out of 8
80 minutes

The slidedeck was presented by the author in this YouTube Video.

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.



"I was designing the first comprehensive type system for references in an OO language (ALGOL W). My goal was to ensure that all use of references should be absolutely safe. But I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years."

— Tony Hoare. *Null References: The Billion Dollar Mistake*. `https://jttu.net/hoare2009null`. [Online; accessed 22-09-2024]. Aug. 2009

- Fail Fast vs Fail Safe
- Alternatives to Returning NULL
- Alternatives to Checking for NULL
- Alternatives to Storing NULL
- Object Thinking
- NULLs in Spring Boot

Chapter #1:

Fail Fast vs Fail Safe



"Over time, more and more errors will fail fast, and you'll see the cost of debugging decrease and the quality of your system improve."

— James Shore. "Fail Fast". In: *IEEE Software* 21.5 (2004). DOI: 10.1109/MS.2004.1331296

Defaults

Fail Safe:

```
1 int size(File file) {
2   if (!file.exists()) {
3     return 0;
4   }
5   return file.length();
6 }
```

Fail Fast:

```
1 int size(File file) {
2   if (!file.exists()) {
3     throw new
    IllegalArgumentException(
4       "The file is absent :("
5     );
6   }
7   return file.length();
8 }
```

The right snippet is more fragile, leading to more errors in runtime, but eventually ... leading to less bugs [1].

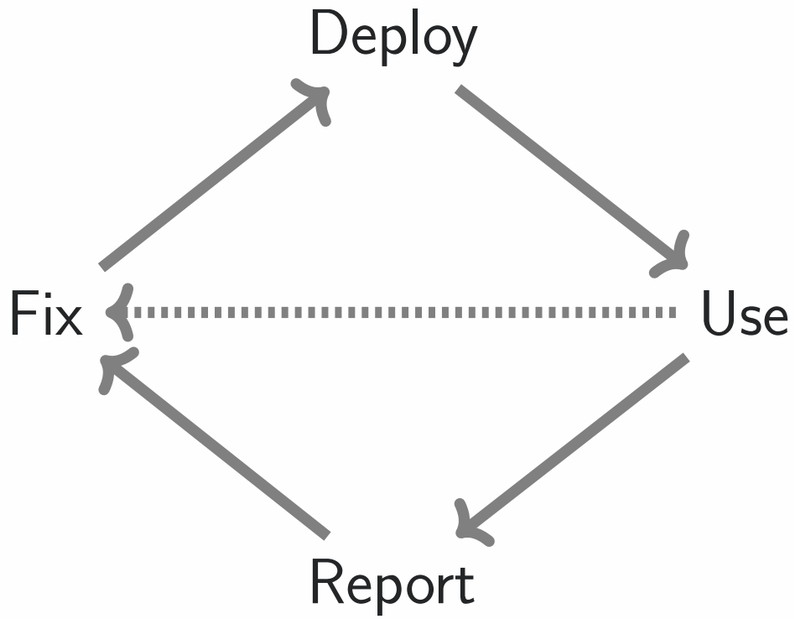
Exception swallowing

```
String read(File file) {
  try {
    return new String(
      Files.readBytes(file)
    );
  } catch (IOException e) {
    e.printStackTrace();
    return ""; // default
  }
}
```

```
String read(File file) {
  try {
    return new String(
      Files.readBytes(file));
  } catch (IOException e) {
    throw new IllegalStateException
  (
      String.format(
        "Can't read file %s", e.
  name()),
      e);
  }
}
```

The right snippet is escalating, while the left one is swallowing.

Software Development Lifecycle



Watch this video from DEVit'2016 conference:
Need It Robust?Make It Fragile!

Chapter #2:

Alternatives to Returning NULL

Returning NULL or raising an error?

```
String nameOfEmployee(int id) {
  if (!em.existsInDb(id)) {
    return null;
  }
  return em.readFromDb(id);
}
```

```
String nameOfEmployee(int id) {
  if (em.existsInDb(id)) {
    throw new EmployeeNotFound(id);
  }
  return em.readFromDb(id);
}
```

The right snippet is “Fail Fast,” that’s why more preferable [4, 3].

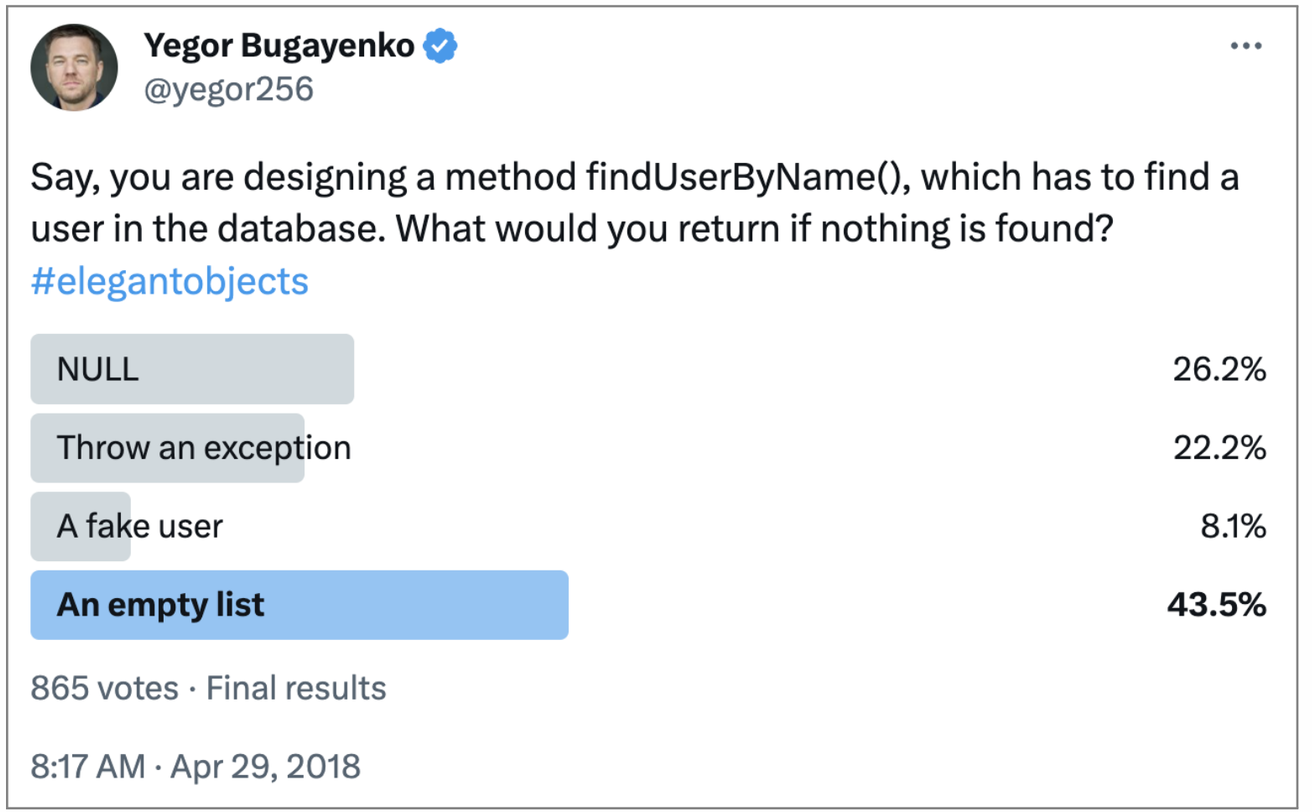
Returning a List or a NULL?

```
String nameOfEmployee(int id) {
  if (!em.existsInDb(id)) {
    return null;
  }
  return em.readFromDb(id);
}
```

```
List<String> nameOfEmployee(int id)
   {
  List<String> names =
    new ArrayList<>(0);
  if (em.existsInDb(id)) {
    names.add(em.readFromDb(id));
  }
  return names;
}
```

There are mo elegant alternatives in most languages, like —Optional— in Java 8+ [2].

[Return List Fake]



Returning a Fake Entity

```
1  Employee employee(int id) {
2    if (!em.existsInDb(id)) {
3      return null;
4    }
5    return new PgEmployee(id);
6  }
7
8  e = employee(42);
9  print(e.id());
10 print(e.salary());
```

```
1  Employee employee(int id) {
2    if (!em.existsInDb(id)) {
3      return FakeEmployee(id);
4    }
5    return new PgEmployee(id);
6  }
7
8  e = employee(42);
9  print(e.id());
10
11 print(e.salary());
```

Chapter #3:

Alternatives to Checking for NULL

null-coalescing operator in C#

```
int? sizeOf(File f) {
  if (!f.exists()) {
    return null;
  }
  return f.size();
}

int? s = sizeOf(f);
if (s == null) {
  s = 0;
}
```

```
int? sizeOf(File f) {
  if (!f.exists()) {
    return null;
  }
  return f.size();
}

int s = sizeOf(f) ?? 0;
```

Both snippets are bad design, though. They are workarounds.

&. operator in Ruby

```ruby
def employee(id)
  unless db.exists?(id)
    return nil
  end
  return db.get(id)
end

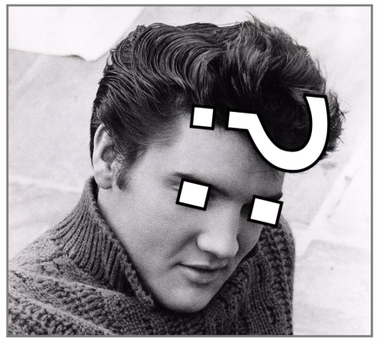
e = employee(42)
puts e.name unless e.nil?
```

```ruby
def employee(id)
  unless db.exists?(id)
    return nil
  end
  return db.get(id)
end

puts employee(42)&.name
```

Actually, the snippets produce different output when the employee is not found. How are they different?

NULL-awareness in Kotlin



```
1 var a: String = "abc"
2 a = null // compilation error
3
4 var b: String? = "abc"
5 b = null // no error here
6
7 println(b?.length) // prints what?
8 println(b?.length ?: -1) // Elvis operator
```

Chapter #4:

Alternatives to Storing NULL

Immutable objects

```
1  class Employee {
2    private String name = null;
3    void setName(String n) {
4      this.name = n;
5    }
6  }
7
8  e = new Employee();
9  e.setName("Jeff");
```

```
1  class Employee {
2    private final String name;
3    Employee(String n) {
4      this.name = n;
5    }
6    Employee withName(String n) {
7      return new Employee(n);
8    }
9  }
10
11 e1 = new Employee();
12 e2 = e1.withName("Jeff");
```

Chapter #5:

Object Thinking

Pay respect to your objects!

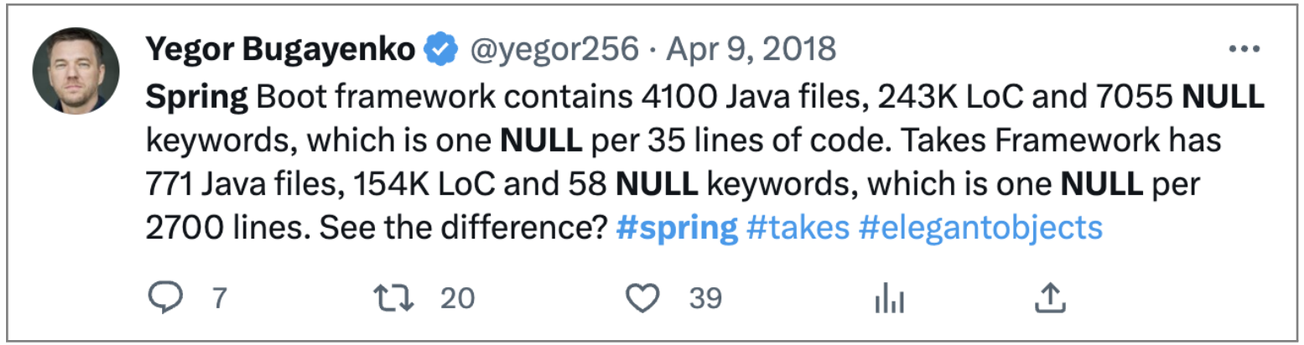
```
1 d = getDepartment(42);
2 e = d.getEmployee("Jeff");
3 if (e != null) {
4   printf("Hello, %s", e.name());
5 }
```

```
1 - Hello, is it the department no
    .42?
2 - Yes.
3 - Let me talk to your employee "
    Jeff".
4 - Hold the line please...
5 - Hello.
6 - Are you NULL?
```



Chapter #6:

NULLs in Spring Boot



You can do you own analysis of existing Java open source GitHub repositories to see how often their developers use —null— keyword.

The Takes framework is here: yegor256/takes.

References

[1] Yegor Bugayenko. *Need Robust Software? Make It Fragile*. https://www.yegor256.com/150825.html. [Online; accessed 22-09-2024]. Aug. 2015.

[2] Yegor Bugayenko. *One More Recipe Against NULL*. https://www.yegor256.com/180522.html. [Online; accessed 22-09-2024]. May 2018.

[3] Yegor Bugayenko. *Throwing an Exception Without Proper Context Is a Bad Habit*. https://www.yegor256.com/151201.html. [Online; accessed 22-09-2024]. Dec. 2015.

[4] Yegor Bugayenko. *Why NULL Is Bad?* https://www.yegor256.com/140513.html. [Online; accessed 22-09-2024]. May 2014.

[5] Tony Hoare. *Null References: The Billion Dollar Mistake*. https://jttu.net/hoare2009null. [Online; accessed 22-09-2024]. Aug. 2009.

[6] James Shore. "Fail Fast". In: *IEEE Software* 21.5 (2004). DOI: 10.1109/MS.2004.1331296.